

VISIÓN ARTIFICIAL. ENTREGA FINAL



Autor:

Mohammed Amrou Labied Nasser
ma.labiednasser@um.es

Tutor:

Alberto Ruiz Garcia

Grupo y subgrupo: 1.1

Fecha de entrega:

27-05-2026

Índice

1	Introducción	3
2	Calibración	4
2.1	Marco Teórico: Modelo Pinhole y Calibración	4
2.1.1	El Modelo Pinhole: Proyección de R^3 a R^2	4
2.1.2	Parámetros Intrínsecos y Matriz K	4
2.1.3	Distorsión y Campo de Visión (FOV)	5
2.2	Calibración: Resultados	5
2.2.1	Obtención de la matriz de calibración K	5
2.2.2	Verificación del modelo y corrección de distorsión	6
2.2.3	Determinación del campo visual FOV	7
2.2.4	Software de medición	9
3	Análisis de tráfico	12
3.1	Metodología y desarrollo del código	12
3.2	Primeros resultados y limitaciones del sistema	13
3.3	El problema de los vehículos parados	14
4	Detector de movimiento	17
4.1	Arquitectura de procesamiento: Modelo Pipeline	17
4.2	Metodología y desarrollo del código	17
4.3	Clasificación de sujetos y control de confianza	18
4.4	Módulo de anonimización y protección de la privacidad	19
4.5	Gestión de notificaciones y almacenamiento	20
5	Clasificador	21
5.1	Arquitectura del Sistema y Aplicación Principal	21
5.1.1	Carga Dinámica y Precomputación	21
5.1.2	Interfaz de Usuario e Interacción	21
5.2	Interfaz Común de Clasificación	22
5.3	Método 1: Clasificación mediante Embeddings	22
5.3.1	Fundamento Técnico	22
5.3.2	Cálculo de Similitud Coseno	22
5.3.3	Visualización Comparativa	22
5.4	Método 2: Reconocimiento de gestos	23
5.4.1	Detección de la mano	23
5.4.2	Análisis de Procrustes y Alineación	23
5.4.3	Cálculo del Score y Visualización	24
5.5	Método 3: Coincidencia de puntos clave mediante SIFT	24
5.5.1	Extracción de Características y Descriptores	24
5.5.2	Proceso de Clasificación	25
5.5.3	Visualización de Correspondencias	25
6	Deep Learning	26
7	Rectificación	27
7.1	Marco teórico: Homografías	27

7.2	Descripción del script	28
7.3	Resultado sobre coins.png	28
7.4	Verificación con imágenes propias	29
7.5	Limitaciones	31
8	OPCIONAL: Carnet UMU	32
9	OPCIONAL LK: Dirección de movimiento y velocidad angular	33
9.1	Marco teórico: Lucas-Kanade	33
9.2	Estimación de la dirección de movimiento	33
9.3	Estimación de la velocidad angular	34
9.4	Script lk_motion.py	35
10	OPCIONAL: Colmap vs Vggt	36
11	OPCIONAL: Realidad Aumentada, desplazamiento de una pelota	37
12	Conclusión	38

1. Introducción

Esta memoria recoge la resolución de los ejercicios propuestos para la entrega parcial de la asignatura de Visión Artificial. El objetivo general ha sido aplicar las técnicas de visión aprendidas sobre problemas reales como calibración de cámara, aplicación de filtros, detección de eventos, reconocimiento, técnicas de *Deep Learning*, etc.

En concreto, la memoria se organiza en los siguientes bloques:

- **Calibración:** El objetivo es ajustar nuestra cámara con múltiples imágenes de un tablero de ajedrez (chessboard) para la estimación del campo visual horizontal y vertical, y desarrollar de un software de medición mediante superposición de una cuadrícula a distancia seleccionable por el usuario.
- **Análisis de tráfico:** Analizar una secuencia de carretera para contar vehículos por sentido y por unidad de tiempo, junto con la generación de gráficas para detectar periodos de mayor intensidad.
- **Detector de movimiento:** Detección de movimiento en una región de interés marcada manualmente, almacenamiento de secuencias de evento, envío de notificaciones y filtrado por categoría de interés, incluyendo anonimización cuando aparecen personas.
- **Clasificador:** Aplicación de reconocimiento de imágenes con soporte para múltiples métodos intercambiables (embedding, reconocimiento de manos, SIFT) para la clasificación de distintos modelos.
- **DL:** Entrenamiento de un modelo con datos propios a través de *Deep Learning*, para la detección de patrones.
- **Rectificación:** Corrección de la perspectiva de imágenes mediante homografías permitiendo medidas entre los objetos, con verificación sobre imágenes propias.
- **OPCIONAL Carnet UMU:** Detección y localización del carnet universitario en escena y posterior intercambio de la foto presente.
- **OPCIONAL LK Avanzado:** Estimación del flujo óptico con Lucas-Kanade para determinar la dirección de movimiento y la velocidad angular de la cámara.
- **OPCIONAL Reconstrucción 3D:** Reconstrucción de un objeto real a partir de vídeo usando *Colmap*, comparada con la reconstrucción basada en deep Learning de *vgg*.
- **OPCIONAL movimiento en Realidad Aumentada:** Efecto de realidad aumentada en el que una esfera virtual rueda sobre el plano de un marcador planar hacia la posición marcada con el ratón, aplicando calibración, estimación de pose y homografía.

El documento presenta para cada parte el fundamento técnico, la metodología seguida y los resultados principales, con especial atención a la trazabilidad entre teoría y comportamiento observado en las pruebas.

2. Calibración

2.1. Marco Teórico: Modelo Pinhole y Calibración

La calibración de la cámara es el proceso fundamental para determinar los parámetros que rigen la formación de la imagen. El objetivo principal es obtener la relación matemática exacta entre los puntos del mundo real y su proyección en el plano de píxeles del sensor, lo que nos permite realizar mediciones métricas precisas.

2.1.1. El Modelo Pinhole: Proyección de R^3 a R^2

Para modelar este proceso de calibración, utilizamos la idealización de la **cámara Pinhole**. Geométricamente, esto supone una transformación que reduce la dimensionalidad del entorno: pasamos del espacio euclídeo tridimensional (R^3) al plano bidimensional de la imagen (R^2).

Como se ilustra en la Figura 1, un punto $P = (X, Y, Z)$ en el espacio proyecta un rayo que atraviesa el centro óptico para situarse en el plano de imagen. Esta proyección se basa en la **semejanza de triángulos**, donde las coordenadas del punto proyectado en el plano de la imagen son proporcionales a sus coordenadas en el espacio tridimensional divididas por su profundidad Z .

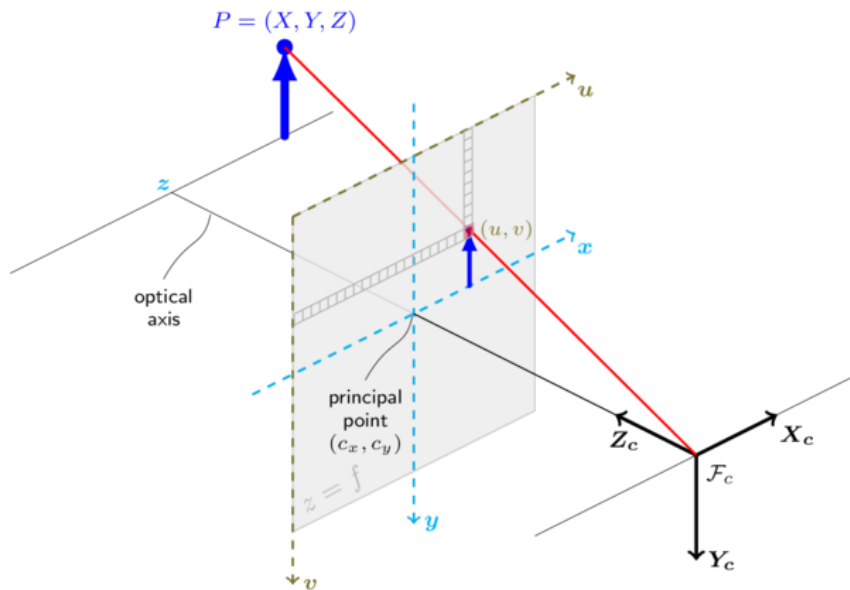


Figura 1: Representación del modelo Pinhole: proyección perspectiva de un punto P del espacio tridimensional sobre el plano de imagen a través del centro óptico.

2.1.2. Parámetros Intrínsecos y Matriz K

En una cámara, esta proyección ideal debe ajustarse para considerar que trabajamos con píxeles y que el eje óptico no coincide necesariamente con la esquina de la imagen. Por ello, un punto 3D se proyecta sobre la imagen siguiendo estas ecuaciones:

$$u = f_x \frac{X}{Z} + c_x, \quad v = f_y \frac{Y}{Z} + c_y,$$

donde f_x y f_y son las distancias focales expresadas en píxeles, y (c_x, c_y) representan el **punto principal** (las coordenadas donde se situa el foco de la cámara). Estos parámetros se agrupan en la llamada **matriz intrínseca** (K):

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}.$$

2.1.3. Distorsión y Campo de Visión (FOV)

En las cámaras reales, la proyección de los rayos de luz no es puramente rectilínea debido a las propiedades físicas de las lentes. Esta desviación respecto al modelo ideal se manifiesta como **distorsión radial y tangencial**. Mientras que la distorsión radial desplaza los puntos de la imagen hacia el centro (efecto cojín) o hacia el exterior (efecto barril), la distorsión tangencial surge de la falta de paralelismo entre la lente y el plano del sensor. La estimación de estos coeficientes permite rectificar la imagen original, recuperando la linealidad del modelo pinhole necesaria para realizar proyecciones métricas precisas.

Una vez caracterizada la distancia focal mediante el proceso de calibración, se puede determinar la apertura angular de la cámara o **Field of View (FOV)**. Este parámetro establece la relación trigonométrica entre las dimensiones del sensor en píxeles ($w \times h$) y la focal estimada (f_x, f_y), definiendo la extensión de la escena que es capturada por el sensor:

$$FOV_h = 2 \arctan \left(\frac{w}{2f_x} \right) \quad FOV_v = 2 \arctan \left(\frac{h}{2f_y} \right).$$

El FOV es una magnitud crítica para la fiabilidad del sistema que vamos a diseñar. Al conocer este ángulo, es posible garantizar que la cuadrícula métrica virtual se proyecte con una perspectiva idéntica a la de la cámara. Esta coherencia geométrica es la que permite transformar desplazamientos medidos en la matriz de píxeles en distancias reales en el mundo físico, asegurando que la escala de la rejilla sea consistente con la profundidad del plano de trabajo.

2.2. Calibración: Resultados

2.2.1. Obtención de la matriz de calibración K

Para llevar a cabo la calibración empírica de la cámara, se han seguido las directrices del procedimiento detallado en el archivo **README** del directorio **code/calibrate**. El proceso requiere la captura de múltiples fotografías de un patrón geométrico. En nuestro caso, se ha empleado un tablero de ajedrez con un patrón de esquinas interiores de 7×7 . Es fundamental que estas capturas abarquen diversas perspectivas e inclinaciones, ya que las imágenes puramente frontales carecen de la deformación perspectiva necesaria para resolver el sistema de ecuaciones.

Como consideración técnica crucial previa al cálculo de parámetros, todas las imágenes capturadas del patrón fueron redimensionadas a una resolución exacta de 270×480 píxeles y teniendo el dispositivo móvil siempre con la misma orientación. Este paso asegura que los parámetros estimados estén perfectamente escalados y sean directamente aplicables a las imágenes de nuestro entorno de trabajo final. Adicionalmente, esta reducción de tamaño

facilita la tarea del algoritmo detector de esquinas, evitando fallos comunes asociados a imágenes de muy alta resolución.

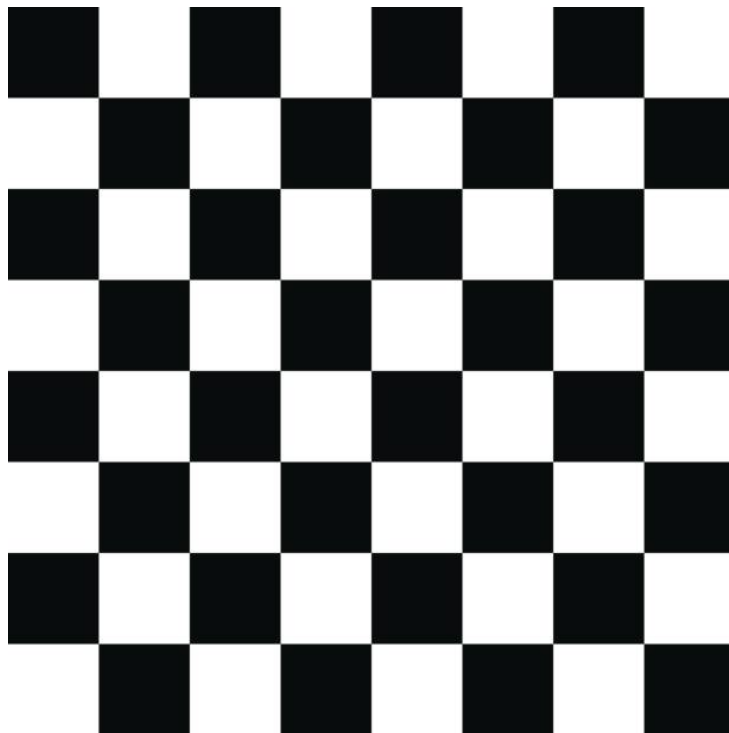


Figura 2: Tablero de ajedrez elemental para la calibración.

Mediante la ejecución del script de calibración (`calibrate.py`), el sistema procesa el conjunto de imágenes y minimiza el error de reproyección global, volcando los resultados en el fichero `calib.txt`. Extrayendo los parámetros numéricos de este archivo, las distancias focales estimadas en píxeles y las ubicaciones del punto principal resultan ser

$$f_x \approx 359,25$$

$$f_y \approx 358,86$$

$$c_x \approx 132,76$$

$$c_y \approx 234,85$$

Por tanto, la matriz intrínseca de calibración K obtenida para nuestra cámara se define como: Extrayendo los parámetros numéricos de este archivo, se obtienen:

$$K \approx \begin{bmatrix} 359,25 & 0 & 132,76 \\ 0 & 358,86 & 234,85 \\ 0 & 0 & 1 \end{bmatrix}.$$

Estos valores constituyen la base geométrica que se integrará posteriormente en el software de medición para proyectar correctamente la cuadrícula tridimensional sobre la imagen.

2.2.2. Verificación del modelo y corrección de distorsión

Una vez calculada la matriz K y los coeficientes de distorsión radial, resulta imprescindible validar la precisión del ajuste. El primer método de validación consiste en proyectar

los puntos tridimensionales ideales del tablero sobre las imágenes originales utilizando los parámetros recién calculados, mediante el script `verify.py`.

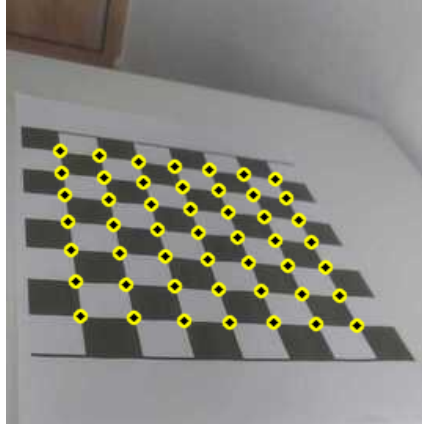


Figura 3: Verificación de la calibración. Los puntos ideales calculados matemáticamente (marcas amarillas) se superponen con alta precisión sobre las esquinas reales del patrón en la fotografía.

Como se puede observar en el ejemplo de la Figura 3, los puntos proyectados matemáticamente coinciden casi a la perfección con las intersecciones físicas de los cuadros del tablero. Esta coincidencia visual confirma que la estimación de las focales y del centro óptico es robusta y modela correctamente la cámara.

El segundo paso de la validación evalúa los coeficientes de distorsión radial estimados (`undistort1.py`). Las lentes físicas tienden a curvar las líneas rectas, un efecto que se acentúa en la periferia de la imagen.

La Figura 4 muestra el resultado tras aplicar la función de des-distorsión (`undistort`). Aunque la lente de nuestra cámara presenta una distorsión radial moderada, se aprecia cómo el algoritmo aplica una transformación espacial en las esquinas de la imagen (creando los bordes negros curvos). Esto garantiza la rectificación del espacio, asegurando que las líneas físicamente rectas en el entorno tridimensional se mantengan como líneas rectas en el plano de la imagen bidimensional, un requisito indispensable para la metrología visual.

2.2.3. Determinación del campo visual *FOV*

El campo de visión o *Field of View* (FOV) es una propiedad angular fundamental que define la amplitud de la escena que el sensor es capaz de capturar. En nuestra aplicación de medición, el conocimiento preciso del FOV es crítico, ya que actúa como el nexo de unión entre la resolución de la imagen en píxeles y las dimensiones angulares del mundo real. Sin un FOV correctamente calibrado, la escala de la cuadrícula virtual no guardaría una relación de proporcionalidad con el plano de trabajo físico, invalidando cualquier intento de medición.

Trabajando en esta entrega con una resolución fija de 270×480 píxeles y empleando las distancias focales obtenidas en la calibración ($f_x \approx 359,25$, $f_y \approx 358,86$), calculamos los ángulos de apertura mediante las relaciones trigonométricas del modelo pinhole:

$$FOV_h = 2 \arctan \left(\frac{270}{2 \cdot 359,25} \right) \approx 41,2^\circ,$$

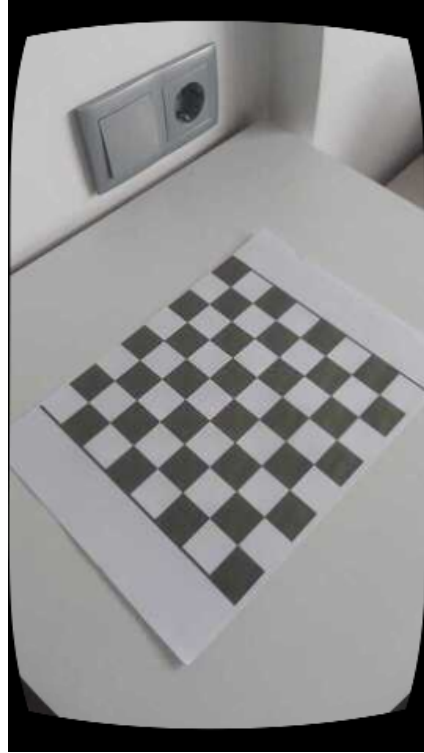


Figura 4: Resultado tras aplicar la rectificación de distorsión radial. La imagen se deforma inversamente para compensar el abombamiento introducido por la lente física.

$$FOV_v = 2 \arctan \left(\frac{480}{2 \cdot 358,86} \right) \approx 67,6^\circ.$$

Es fundamental destacar la aplicación de estos resultados en la herramienta de medición interactiva. Debido a que el script calcula la distancia focal operativa basándose en el ancho de la imagen ($w = 270$ px), el usuario deberá ajustar el deslizador del FOV en la interfaz al valor horizontal a $41,2^\circ$ siendo esta una constante para imágenes de 270×480 píxeles .

Ejemplo de aplicación práctica: Posición de una cámara

Para ilustrar la importancia del FOV en la planificación de sistemas de visión, calcularemos la altura mínima a la que debería situarse nuestra cámara para abarcar completamente una pista de tenis estándar (cuyas dimensiones para dobles son $23,77 \times 10,97$ metros).

Dado que nuestra imagen está en formato retrato o vertical (270×480), la optimización del espacio visual requiere orientar la cámara de forma que el lado más largo de la pista ($23,77$ m) coincida con el FOV más amplio del sensor ($FOV_v = 67,6^\circ$) y el ancho de la pista ($10,97$ m) coincida con el FOV más estrecho ($FOV_h = 41,2^\circ$). Aplicando trigonometría básica, calculamos la altura h necesaria para capturar cada eje:

- Altura necesaria para cubrir el largo ($23,77$ m) usando el FOV_v :

$$\tan \left(\frac{67,6^\circ}{2} \right) = \frac{23,77/2}{h_{largo}} \Rightarrow h_{largo} = \frac{11,885}{\tan(33,8^\circ)} \approx 17,75 \text{ m}$$

- Altura necesaria para cubrir el ancho (10,97 m) usando el FOV_h :

$$\tan\left(\frac{41,2^\circ}{2}\right) = \frac{10,97/2}{h_{ancho}} \Rightarrow h_{ancho} = \frac{5,485}{\tan(20,6^\circ)} \approx 14,59 \text{ m}$$

Para garantizar que la pista completa quede enmarcada dentro del campo visual, debemos seleccionar la mayor de las dos alturas calculadas. Por tanto, la cámara debería instalarse a una altura mínima de **17.75 metros**. Este análisis geométrico demuestra cómo los parámetros de calibración permiten diseñar y validar la disposición física de un sistema de visión antes de su instalación.

2.2.4. Software de medición

Para poner en práctica los parámetros calculados, se ha desarrollado una aplicación interactiva que superpone una cuadrícula tridimensional de medida sobre la imagen de 270×480 píxeles (mirar la carpeta `code/calibrate/cuadrícula.py`). Esta cuadrícula actúa como un sistema de referencia métrico virtual sobre la escena.

A nivel de implementación, el programa lee automáticamente los parámetros de calibración (f_x , c_x , c_y) desde el archivo `calib.txt`. A través de la librería `umucv.util`, se ha dotado a la interfaz de una serie de deslizadores interactivos (*sliders*) que permiten al usuario ajustar dinámicamente la perspectiva de la cuadrícula. Estos controles modifican variables clave: el campo de visión (FOV), la profundidad o distancia de la cámara al plano frontal (Z), la altura de la cámara respecto al suelo (A) y el desplazamiento lateral de la rejilla (X).

Internamente, cada vez que se mueve un deslizador, el script recalcula la focal operativa basándose en el ancho de la imagen y el FOV seleccionado. Seguidamente, utiliza una función de proyección basada en el modelo Pinhole para mapear las líneas del espacio 3D a sus correspondientes coordenadas 2D en la imagen (`proj_points`), dibujando así tanto el plano frontal como el plano del suelo de forma que respeten la perspectiva real de la cámara.

La característica más relevante de este software es su capacidad para estimar distancias directas sobre los objetos. Para la implementación de esta funcionalidad, nos hemos inspirado en la arquitectura del script `mouse.py` estudiado durante las sesiones prácticas. Mediante el uso de una estructura `deque`, el sistema captura los eventos del ratón (clics izquierdos) para registrar hasta dos puntos en la pantalla. Una vez que el usuario marca dos puntos, el programa calcula la distancia euclidiana en píxeles que los separa. Posteriormente, combinando la distancia focal operativa (f) y la profundidad del plano (Z) configurada en el deslizador, aplica relaciones trigonométricas de semejanza para transformar esa magnitud temporal en una medida real aproximada en centímetros ($d_{cm} = 100 \cdot d_{pix} \cdot \frac{Z}{f}$).

Ejemplo de uso práctico y validación

Para comprobar el funcionamiento conjunto de la calibración y el software desarrollado, se ha realizado una medición de prueba sobre un objeto de la escena: una botella de agua situada sobre una cajonera. En la Figura 6 se muestra el resultado del programa durante este proceso.

El procedimiento seguido para obtener una medición precisa consta de dos fases críticas de ajuste:



Figura 5: Interfaz del software realizando la medición de la altura de la botella. Se observan los textos superpuestos con la configuración geométrica y el resultado final.

1. **Configuración del FOV:** Tal como se ha justificado teóricamente, es imperativo alinear la matemática del script con la calibración real. El deslizador del FOV se ajustó a $41,0^\circ$ (valor muy próximo al $41,2^\circ$ teórico). Al hacerlo, el programa calcula una distancia focal operativa de $f = 361$ px, coincidiendo con alta precisión con el parámetro $f_x \approx 359$ px de nuestro archivo `calib.txt`.
2. **Ajuste de la profundidad (Z):** A continuación, se indicó al programa a qué distancia física se encuentra el objeto respecto a la lente de la cámara. Mediante el deslizador correspondiente, se estableció una profundidad de $Z = 1,5$ m. Este ajuste se comprueba visualmente asegurando que la línea gruesa de la cuadrícula coincida con la base de los objetos sobre el mueble, validando así la perspectiva tridimensional.

Una vez configurado el entorno geométrico, el usuario procede a medir interactuando con el ratón sobre la imagen. En este ejemplo, se han marcado dos puntos: uno en la base y otro en el tapón de la botella. El software calcula automáticamente la distancia euclidiana entre ambos clics, registrando un valor de $d_{pix} = 80,0$ píxeles.

Cálculo analítico del resultado: Para transformar esta magnitud bidimensional en una medida métrica real, el código aplica la relación geométrica de semejanza de triángulos dictada por el modelo Pinhole. La fórmula implementada en el script es:

$$d_m = \frac{d_{pix} \cdot Z}{f}$$

Sustituyendo por los valores extraídos de la interfaz:

$$d_m = \frac{80,0 \cdot 1,5}{361} \approx 0,3324 \text{ metros}$$

Finalmente, el programa multiplica este resultado por 100 para presentar la información en centímetros ($d_{cm} = 100 \cdot d_m$). Esto arroja un valor final de **33.2 cm**, tal y como se refleja en la etiqueta amarilla de la imagen.

Para validar empíricamente la precisión del sistema, se procedió a contrastar la estimación del software con la dimensión física del objeto en el entorno real. La medición manual de la botella arrojó una altura de 32.5 cm. Al comparar esta magnitud empírica con los 33.2 cm obtenidos mediante nuestro programa, se observa una desviación absoluta de apenas 0.7 cm, lo que se traduce en un error relativo aproximado del 2.15 %. Esta reducida tasa de error confirma la fiabilidad de la matriz de calibración calculada y demuestra la viabilidad del modelo geométrico implementado para aplicaciones de metrología visual básica



Figura 6: Medida real de la botella de agua.

3. Análisis de tráfico

El objetivo principal de este ejercicio es desarrollar un sistema de visión artificial capaz de monitorizar y cuantificar el flujo de tráfico en una vía. Concretamente, el sistema debe ser capaz de detectar los vehículos en movimiento, seguirlos a lo largo de la pantalla y contar de forma independiente cuántos coches se desplazan hacia la derecha y cuántos hacia la izquierda.

3.1. Metodología y desarrollo del código

Para llevar a cabo esta tarea, hemos tomado como base los scripts proporcionados en la teoría (`code/color/backsub*.py`), los cuales introducen las técnicas de sustracción de fondo en nuestro script (mirar `code/trafico/trafico.py`).

El núcleo de nuestro detector se basa en el algoritmo **MOG2** que es un método de segmentación de fondo que modela cada píxel como una mezcla de distribuciones gaussianas. De forma simplificada, el algoritmo “aprende” qué colores son los habituales en cada zona de la imagen y detecta como “movimiento” cualquier píxel que cambie drásticamente de color, marcándolo de color blanco en una máscara binaria.

Para optimizar la detección y evitar falsos positivos, hemos implementado una mejora fundamental en el código:

- **Región de Interés (ROI):** Si aplicamos el sustractor a toda la imagen, elementos como el flujo del río o el movimiento de los pájaros generan ruido constante. Para evitarlo, creamos una máscara negra geométrica y delimitamos la zona activa exclusivamente al área del puente. Mediante una operación `bitwise_and`, el algoritmo ignora todo lo que ocurra fuera de la carretera.

Por otro lado, el sistema de seguimiento y conteo direccional (*Tracking*) constituye uno de los pilares más importantes del código. Dado que el algoritmo MOG2 no reconoce vehículos, sino únicamente agrupaciones de píxeles en movimiento, es necesario dotar a cada objeto de una identidad temporal. Para lograrlo, calculamos el centroide matemático de la caja delimitadora (*bounding box*) de cada coche detectado en el fotograma actual.

A continuación, implementamos un algoritmo de seguimiento espacial. El programa almacena un historial con las coordenadas de los vehículos del fotograma anterior y evalúa la distancia euclidiana (`np.hypot`) respecto a los centros actuales. Si la distancia entre un punto antiguo y uno nuevo es inferior a un umbral predefinido (en nuestro caso, 50 píxeles), el sistema infiere que se trata del mismo vehículo avanzando por la vía.

Una vez emparejado el vehículo con su posición pasada, se evalúa la lógica de cruce: se comprueba si la coordenada X anterior (`cx_ant`) se encontraba a un lado de la línea vertical central y si la coordenada actual (`cx_act`) la ha superado. Dependiendo del sentido de este desplazamiento, se incrementa el contador del carril derecho o izquierdo. Finalmente, para aportar robustez al sistema, se ha implementado una variable de estado (`contado`). Esta etiqueta booleana previene conteos múltiples causados por la inestabilidad o “vibración” de las cajas delimitadoras, garantizando que, aunque un vehículo fluctúe temporalmente sobre la línea divisoria, sume un único punto en el marcador general.

3.2. Primeros resultados y limitaciones del sistema

En las pruebas iniciales con tráfico fluido, el sistema demuestra un rendimiento excelente. Como se puede observar en la Figura 7, cuando los vehículos mantienen una distancia de seguridad adecuada entre ellos, el algoritmo MOG2 junto con las operaciones morfológicas aísla perfectamente cada coche, envolviéndolo en su propia caja delimitadora (*bounding box*) y contándolos de forma precisa.

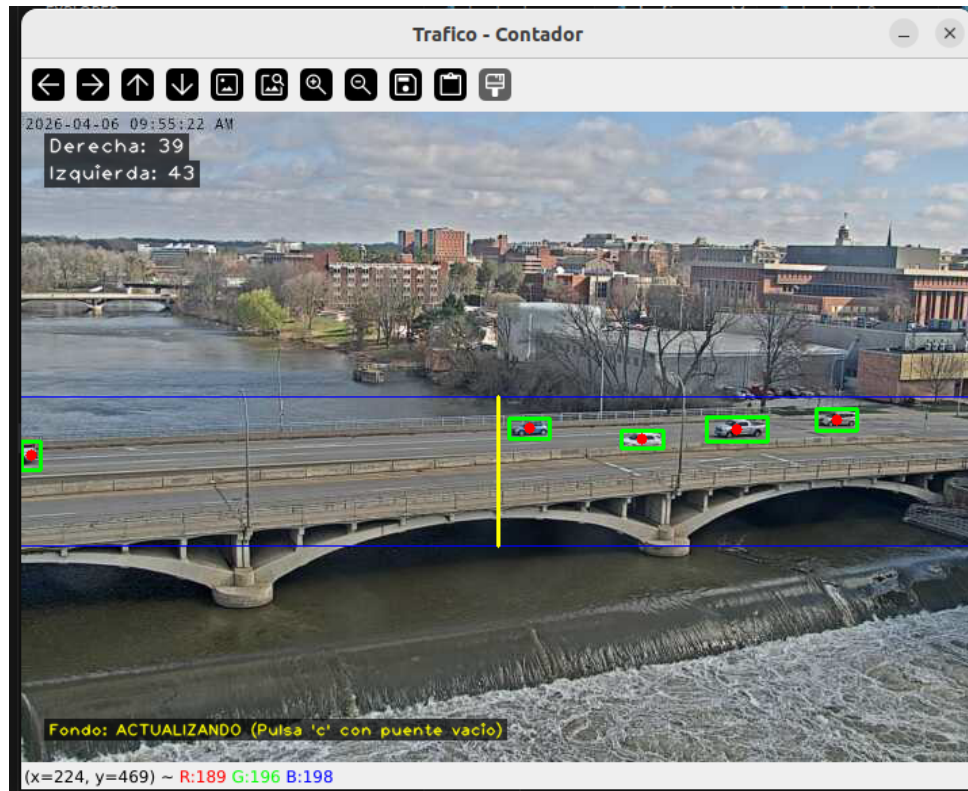


Figura 7: Detección correcta de vehículos circulando con separación adecuada.

Sin embargo, el sistema presenta una vulnerabilidad conocida en visión artificial clásica: el problema de la fusión de contornos u oclusión. Tal y como se ilustra en la Figura 8, cuando dos o más vehículos circulan muy juntos (ya sea en el mismo carril o adelantándose en carriles contiguos), la propia perspectiva de la cámara hace que visualmente sus masas se toquen o superpongan. Como resultado, el algoritmo detecta un único contorno gigante en la máscara de movimiento que engloba a varios coches, provocando que el contador registre un solo vehículo en lugar de varios. Una solución robusta a este problema sería la implementación de un modelo de *Machine Learning* (como redes neuronales especializadas en detección de objetos) capaz de identificar y separar visualmente cada vehículo de forma individual, aunque dicha aproximación queda fuera del alcance de las técnicas clásicas aplicadas en esta práctica.

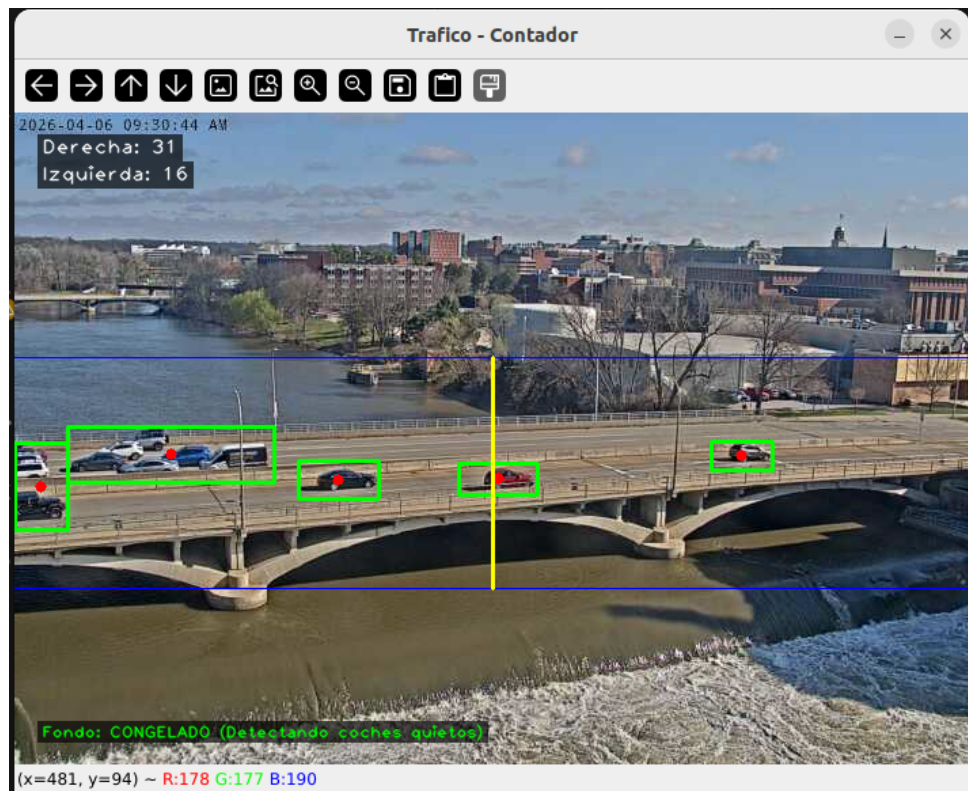


Figura 8: Fallo de detección (fusión de contornos) al circular los vehículos muy próximos.

3.3. El problema de los vehículos parados

Además de la oclusión de contornos, los algoritmos dinámicos de sustracción de fondo presentan un desafío inherente relacionado con su tasa de aprendizaje o adaptación temporal. El algoritmo MOG2 está diseñado para actualizar constantemente su modelo de fondo, una característica esencial para que el sistema se adapte a cambios graduales de iluminación a lo largo del día o al movimiento sutil de las sombras.

Sin embargo, esta misma virtud se convierte en un problema cuando la dinámica del tráfico cambia y los vehículos se detienen por completo en la vía, como ocurre habitualmente durante un atasco. Al permanecer estáticos en la misma posición durante varios fotogramas, los píxeles que conforman el vehículo dejan de presentar variaciones temporales. En consecuencia, el algoritmo interpreta que esos píxeles estáticos ya no representan un objeto en movimiento, sino que constituyen una alteración permanente en la escena, asimilándolos de forma progresiva dentro de su modelo de fondo.

El resultado directo de este proceso de adaptación es que el vehículo estacionado se desvanece de la máscara binaria de movimiento, perdiendo su contorno y evadiendo por completo el sistema de detección. En la Figura 9 se puede observar claramente este fenómeno: a pesar de existir una retención evidente con varios coches detenidos en el carril izquierdo, el sistema omite dibujarlos, puesto que el modelo de fondo se ha actualizado recientemente y ha integrado sus siluetas como si fuesen parte del asfalto.

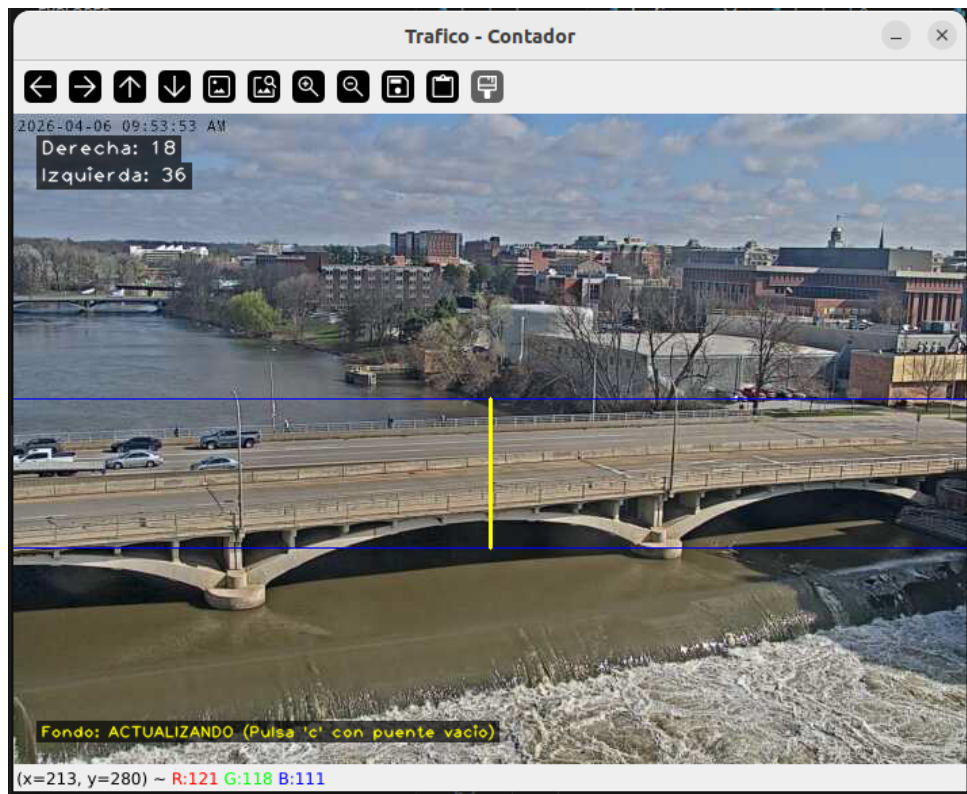


Figura 9: Los vehículos detenidos se integran en el fondo y dejan de ser detectados.

Para solucionar este inconveniente, nos hemos inspirado en el funcionamiento del script teórico `backsub.py`, implementando un mecanismo de “congelación de fondo”. El sistema permite capturar y guardar un fotograma donde el puente se encuentra completamente vacío mediante la pulsación de la tecla ‘c’. Técnicamente, esta acción reduce la variable de aprendizaje (`learningRate`) a 0.

Al anular la tasa de aprendizaje, obligamos al algoritmo MOG2 a detener su proceso de actualización; la imagen del puente vacío queda anclada en la memoria como el fondo absoluto y definitivo. A partir de ese momento, el sistema realiza la sustracción en cada fotograma contra ese fondo inalterable. El impacto de esta modificación se documenta en la Figura 10: al disponer de un fondo congelado libre de vehículos, cualquier coche que ingrese a la escena y se detenga mantendrá una discrepancia de píxeles constante respecto al fondo original. Esto garantiza que siga siendo detectado y enmarcado indefinidamente, ofreciendo una solución robusta al problema de las retenciones de tráfico utilizando únicamente visión clásica pura aunque esto no evita los errores de la fusión de contornos.

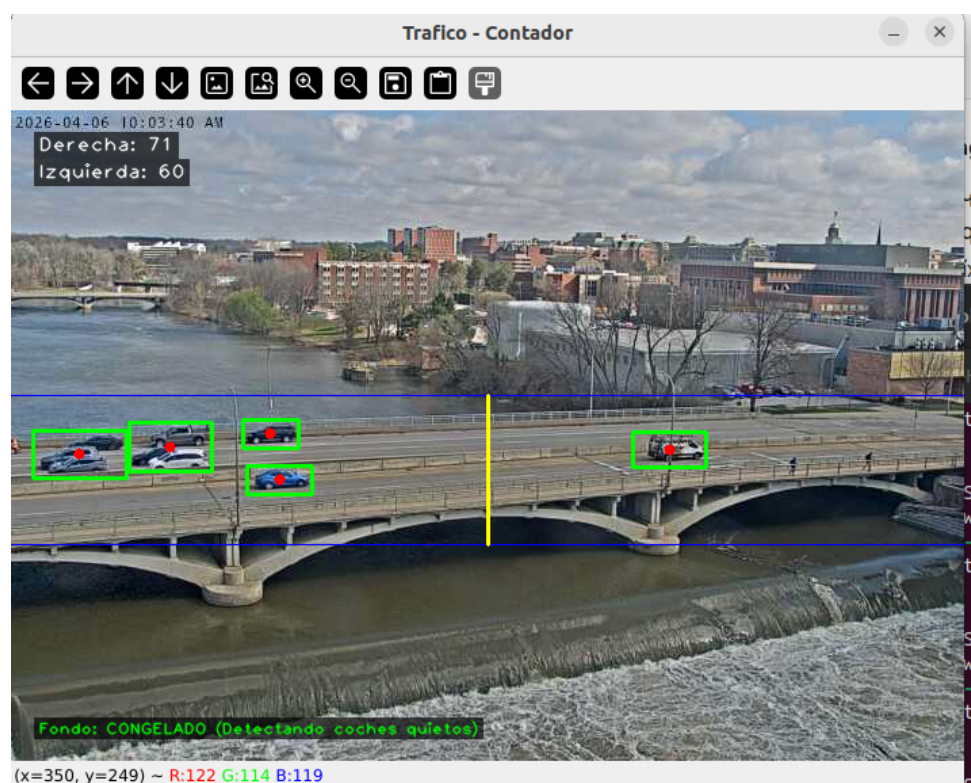


Figura 10: Detección exitosa de vehículos parados mediante la congelación del modelo de fondo.

4. Detector de movimiento

En este apartado se ha implementado un sistema inteligente de vigilancia capaz de detectar movimiento en una región de interés (ROI), clasificar el objeto causante del evento y notificar al usuario de forma selectiva manteniendo la privacidad.

4.1. Arquitectura de procesamiento: Modelo Pipeline

La arquitectura del sistema se articula sobre un modelo de procesamiento en cascada o *pipeline* secuencial, orientado a la eficiencia computacional y la reducción de falsos positivos. El flujo de datos se divide en tres niveles críticos:

1. **Preprocesamiento y delimitación de la ROI:** En la etapa inicial del bucle de procesamiento, el sistema verifica la existencia de una Región de Interés definida por el usuario. Antes de realizar cualquier cálculo de visión artificial, se extrae el sub-segmento de la imagen (**trozo**) directamente del fotograma original. Esta segmentación previa es fundamental, ya que reduce el número de píxeles a procesar en las etapas posteriores, permitiendo una ejecución fluida en hardware con recursos limitados.
2. **Detección de bajo nivel (Segmentación de movimiento):** El sistema aplica el algoritmo **MOG2** (*Mixture of Gaussians*) como se hizo en el ejercicio anterior únicamente sobre el recorte de la ROI obtenido en la fase anterior. Esta etapa identifica variaciones significativas en la intensidad de los píxeles de forma económica en términos de CPU. Solo cuando la masa de movimiento supera un umbral crítico (**motion >500**), se activa la lógica de grabación y el análisis de alto nivel.
3. **Detección de alto nivel (Inferencia y Clasificación):** Condicionado por la detección de movimiento previa, se invoca al modelo de *Deep Learning* **YOLO** para detectar personas. Al limitar la inferencia exclusivamente al área de la ROI y solo durante eventos de actividad confirmada, se optimiza el uso de la unidad de procesamiento y se evita la carga redundante de analizar escenas estáticas.

4.2. Metodología y desarrollo del código

Para el desarrollo los scripts `detector.py` y `botVideo.py` (mirar carpeta `code/detector`), se han tomado como referencia los scripts fundamentales de la asignatura: `roi.py` para la gestión de regiones de interés, `backsub*.py` para las técnicas de sustracción de fondo, `yolo.py` para la integración de modelos de *Deep Learning* y la librería personalizada `bot*.py` para la comunicación con la API de Telegram.

El núcleo del sistema utiliza el algoritmo **MOG2** para modelar el fondo mediante mezclas de distribuciones gaussianas. A diferencia de una resta de fotogramas simple, este método permite que el detector se adapte a cambios de iluminación y descarte variaciones irrelevantes en la escena. El programa monitoriza el número de píxeles activos en la máscara binaria resultante; cuando el recuento supera un umbral de 500 unidades, el sistema infiere la presencia de una perturbación en la ROI.

Para garantizar la robustez del sistema y optimizar la precisión de las notificaciones, se han implementado las siguientes mejoras y lógicas de control:

- **Gestión dinámica de la ROI :** el usuario delimita el área activa. El script extrae este segmento (`trozo`) y lo procesa de forma aislada. Esto no solo ahorra recursos de CPU, sino que evita que movimientos fuera de la zona de interés (como vehículos en la lejanía o vegetación) activen el sistema de forma errónea.
- **Filtros de persistencia temporal:** Para evitar grabaciones espurias causadas por ruido momentáneo, se ha implementado un contador de fotogramas (`motion_run`). El sistema solo inicia la escritura del archivo de vídeo si el movimiento es persistente durante al menos 3 fotogramas consecutivos. Del mismo modo, para evitar cortes abruptos en la captura, la grabación solo finaliza tras una ausencia de movimiento de 20 fotogramas seguidos (`motion_frames_to_stop`), garantizando que el evento se registre íntegramente.
- **Clasificación y Anonimización:** Durante el estado de grabación, el sistema invoca al modelo **YOLO** para validar si el objeto detectado pertenece a la categoría de personas. Si el modelo confirma la detección con una confianza superior a la marcada en el *slider*, se activa una bandera de estado (`person_detected_in_video`). Simultáneamente, se ejecuta un detector de rostros basado en **Haar Cascades** dentro del área delimitada por YOLO. Si se localiza una cara, se aplica un desenfoque gaussiano de 40×40 , asegurando el cumplimiento de la normativa de privacidad en planos cercanos.
- **Finalización y Notificación:** Una vez cerrado el flujo de escritura del vídeo, la función `finalize_and_send` evalúa la validez del archivo. Solo si se cumplen simultáneamente los requisitos de duración mínima (5 segundos) y confirmación de categoría (detección de persona), se procede al envío del archivo a través del bot de Telegram. En caso contrario, el script invoca la función `remove_video` para eliminar el archivo local y preservar el espacio en disco, garantizando que el usuario solo reciba alertas de eventos verificados.

4.3. Clasificación de sujetos y control de confianza

El sistema emplea el modelo pre-entrenado `yolo11n.pt` para la identificación y categorización de los elementos en movimiento. Se ha configurado el modelo para responder exclusivamente a la clase `person` (índice 0 del *dataset* COCO).

Para dotar al sistema de adaptabilidad frente a distintas condiciones ambientales (ruido del sensor, variaciones lumínicas o distancia de los sujetos), se ha integrado una interfaz de control mediante un componente **Slider**. Este elemento permite al usuario ajustar dinámicamente el umbral de confianza (*confidence threshold*) del modelo YOLO en tiempo real. Gracias a este control, es posible filtrar detecciones marginales o falsos positivos, asegurando que solo los eventos con una alta probabilidad de acierto activen el flujo de grabación y notificación, optimizando así los recursos del sistema.

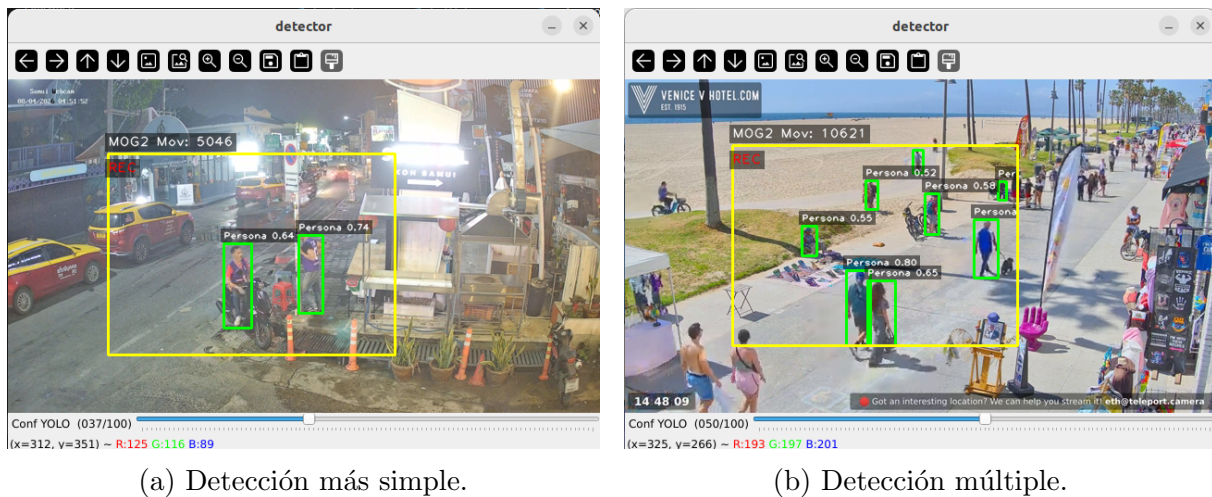


Figura 11: Resultados de inferencia del modelo YOLOv11n mostrando la localización de sujetos y sus respectivos índices de confianza configurados mediante el *slider*.

4.4. Módulo de anonimización y protección de la privacidad

Con el objetivo de garantizar la privacidad en las secuencias capturadas, el sistema integra un proceso de anonimización automática y selectiva sobre las detecciones confirmadas. Tras la localización del sujeto por parte de YOLO, se extrae la región de la imagen delimitada por su caja de detección (*bounding box*).

Sobre dicho recorte, se ejecuta un clasificador **Haar Cascade** de OpenCV especializado en la localización de rostros. Si se identifica una cara (especialmente en planos cortos donde la identidad es reconocible), se procede a la aplicación de un kernel de difuminado (*blur*) agresivo de 40×40 píxeles directamente sobre la región del rostro. Este método asegura que la anonimización sea efectiva sin degradar la información visual del resto de la escena capturada, como se observa en la Figura 12.

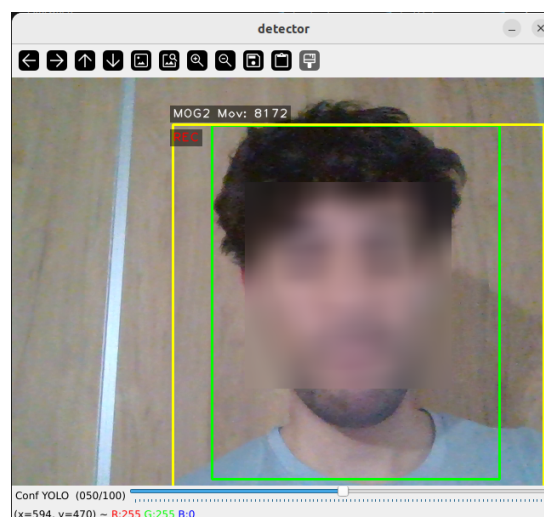


Figura 12: Anonimización efectiva mediante difuminado local del rostro detectado.

4.5. Gestión de notificaciones y almacenamiento

La fase final del ciclo de procesamiento se encarga de la validación de los eventos y la higiene del sistema de archivos, asegurando que solo la información de alta relevancia sea notificada al usuario. Tras finalizar un evento de movimiento, la función `finalize_and_send` evalúa la validez de la secuencia basándose en dos criterios críticos:

- **Criterio de envío:** El vídeo capturado solo se transmite al usuario vía Telegram si cumple una duración mínima de 5 segundos y si se ha confirmado la presencia de una persona en al menos un fotograma de la secuencia.
- **Gestión de recursos:** Mediante la función `finalize_and_send`, el sistema descarta y elimina automáticamente los archivos que no cumplen los requisitos mínimos de duración o que no contienen sujetos de la categoría de interés, optimizando así el almacenamiento local.

5. Clasificador

En este apartado se ha desarrollado una aplicación de reconocimiento de imágenes capaz de identificar objetos o escenas a partir de un conjunto de modelos previamente almacenados. El sistema compara cada fotograma de entrada con dichos modelos utilizando distintos métodos de similitud, mostrando como resultado el modelo más probable junto con medidas asociadas como distancia o confianza.

La aplicación está diseñada de forma modular, permitiendo la incorporación sencilla de nuevos métodos de comparación mediante una interfaz común. Además, se optimiza el rendimiento mediante el preprocesado de los modelos y se contemplan diferentes enfoques de reconocimiento, como el uso de embeddings, comparación de formas o coincidencia de puntos clave.

5.1. Arquitectura del Sistema y Aplicación Principal

El núcleo del proyecto reside en el script `code/clasificador/clasificador.py`, el cual actúa como orquestador del sistema. Su diseño se basa en la flexibilidad, permitiendo al usuario configurar la ejecución mediante dos argumentos de línea de comandos obligatorios:

- **--models:** Especifica el directorio que contiene las imágenes de referencia que el sistema debe reconocer.
- **--method:** Define el módulo de clasificación (ubicado en la carpeta `methods/`) que se empleará para la comparación.

5.1.1. Carga Dinámica y Precomputación

Para lograr una arquitectura extensible, el programa utiliza la carga dinámica de módulos. Esto permite añadir nuevos métodos de clasificación simplemente creando un archivo `.py` en la carpeta correspondiente sin necesidad de modificar el código principal.

Antes de iniciar el bucle de captura, el sistema realiza una etapa de **precomputación**. Recorre cada imagen del directorio de modelos y ejecuta la función `prepare()` del método seleccionado. Esta fase es crítica para el rendimiento, ya que permite extraer y almacenar descriptores complejos (como vectores SIFT o embeddings de MediaPipe) una sola vez, evitando cálculos redundantes durante la ejecución en tiempo real.

5.1.2. Interfaz de Usuario e Interacción

El clasificador no es un sistema estático; permite la gestión de modelos en caliente mediante comandos de teclado durante la ejecución:

- **Tecla 'c' (Capture):** Extrae las características del fotograma actual y las añade instantáneamente a la lista de modelos en memoria. Esto permite al sistema reconocer un nuevo objeto de forma inmediata sin reiniciar la aplicación.
- **Tecla 's' (Save):** Almacena permanentemente en el disco (dentro de la carpeta de modelos) todas aquellas capturas realizadas previamente con la tecla 'c', garantizando la persistencia de los nuevos modelos aprendidos.

5.2. Interfaz Común de Clasificación

Todos los métodos implementados comparten una estructura de clase (*interface*) común, lo que garantiza la compatibilidad con el orquestador:

1. `prepare(img)`: Toma una imagen de modelo y devuelve un diccionario con los datos necesarios para la comparación (descriptores, puntos clave, etc.).
2. `compare(frame, trained_models)`: Toma el fotograma actual de la cámara y lo compara con toda la base de datos precomputada, devolviendo una lista ordenada de resultados por puntuación de confianza.

5.3. Método 1: Clasificación mediante Embeddings

Este método representa el enfoque de *Deep Learning* para el reconocimiento general de objetos y escenas. A diferencia de los métodos geométricos o de textura, este utiliza una red neuronal convolucional para interpretar el contenido semántico de la imagen.

5.3.1. Fundamento Técnico

El módulo emplea el modelo **MobileNet V3 Small**, una arquitectura optimizada para dispositivos con recursos limitados que transforma una imagen de entrada en un vector numérico de alta dimensionalidad llamado *embedding*.

- **Normalización L2**: El modelo se configura con la opción `l2_normalize=True`. Esto asegura que todos los vectores resultantes tengan una norma unitaria, simplificando el cálculo de la similitud matemática.
- **Precomputación (prepare)**: Durante la fase de carga, el método extrae el vector de características de cada imagen de la carpeta de modelos y lo almacena junto con una copia de la imagen original en un diccionario.

5.3.2. Cálculo de Similitud Coseno

La comparación en tiempo real se basa en la **Similitud Coseno** entre el vector del fotograma actual (q) y el vector de cada modelo almacenado (m).

Dado que los vectores están normalizados, la similitud se reduce al producto escalar de ambos:

$$score = q \cdot m$$

Un resultado cercano a 1,0 indica una coincidencia casi total en el contenido visual, mientras que valores bajos sugieren objetos distintos.

5.3.3. Visualización Comparativa

Para facilitar la validación por parte del usuario, el archivo `embedder.py` implementa una lógica de visualización específica mediante la función `_show_best`:

- **Interfaz de doble panel**: El sistema genera una ventana adicional que muestra, lado a lado mediante `np.hstack`, el fotograma “Actual” capturado por la cámara y la imagen del “Modelo” que ha obtenido la puntuación más alta.

- **Retroalimentación de confianza:** Se imprime el valor del *score* directamente sobre la imagen combinada, permitiendo observar la robustez de la red neuronal ante cambios de iluminación o ligeros desplazamientos del objeto.

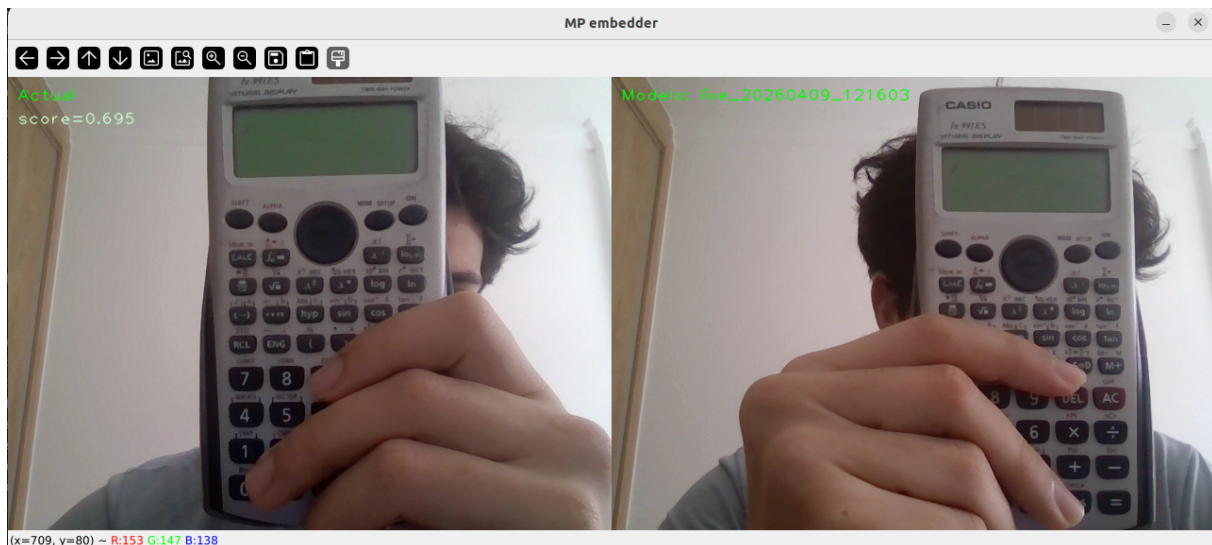


Figura 13: Ejecución del clasificador por embeddings comparando una calculadora científica en tiempo real frente al modelo almacenado.

5.4. Método 2: Reconocimiento de gestos

Este módulo implementa un reconocedor de gestos especializado en la morfología de la mano. A diferencia del método anterior, este enfoque ignora la textura y el color, centrándose exclusivamente en la configuración geométrica de las articulaciones para identificar poses específicas.

5.4.1. Detección de la mano

El sistema utiliza la librería **MediaPipe Hands** para localizar en tiempo real 21 puntos clave (coordenadas x, y) que representan la estructura ósea de la mano.

- **Robustez:** El modelo de MediaPipe permite una detección estable incluso con oclusiones parciales de los dedos.
- **Extracción (prepare):** Para cada modelo de referencia, se extraen estas coordenadas y se almacenan junto a la imagen original, permitiendo una comparación posterior basada únicamente en vectores de puntos.

5.4.2. Análisis de Procrustes y Alineación

Para que la comparación sea efectiva independientemente de dónde se encuentre la mano en la imagen o de su tamaño, se aplica un proceso de normalización y alineación matemática:

1. **Normalización de forma:** Se traslada la mano para que su centroide esté en el origen $(0, 0)$ y se escala dividiendo por su norma, haciendo que la comparación sea invariante a la posición y a la distancia respecto a la cámara.

2. **Alineación de rotación:** Dado que el usuario puede inclinar la mano, el sistema busca la rotación óptima entre la mano actual (q) y el modelo (p). Esto se logra mediante la **Descomposición en Valores Singulares (SVD)** del producto de sus matrices de puntos.
3. **Métrica de Distancia:** Una vez alineadas, se calcula la norma de la diferencia entre ambas configuraciones.

5.4.3. Cálculo del Score y Visualización

La distancia obtenida se transforma en un valor de confianza mediante la fórmula:

$$score = \frac{1,0}{1,0 + distancia}$$

Al igual que en el método de *embeddings*, se ha implementado una visualización de doble panel que permite ver el esqueleto de puntos amarillos sobreimpreso tanto en la mano actual como en el modelo ganador, facilitando la comprensión de por qué el sistema considera que ambas poses son equivalentes.

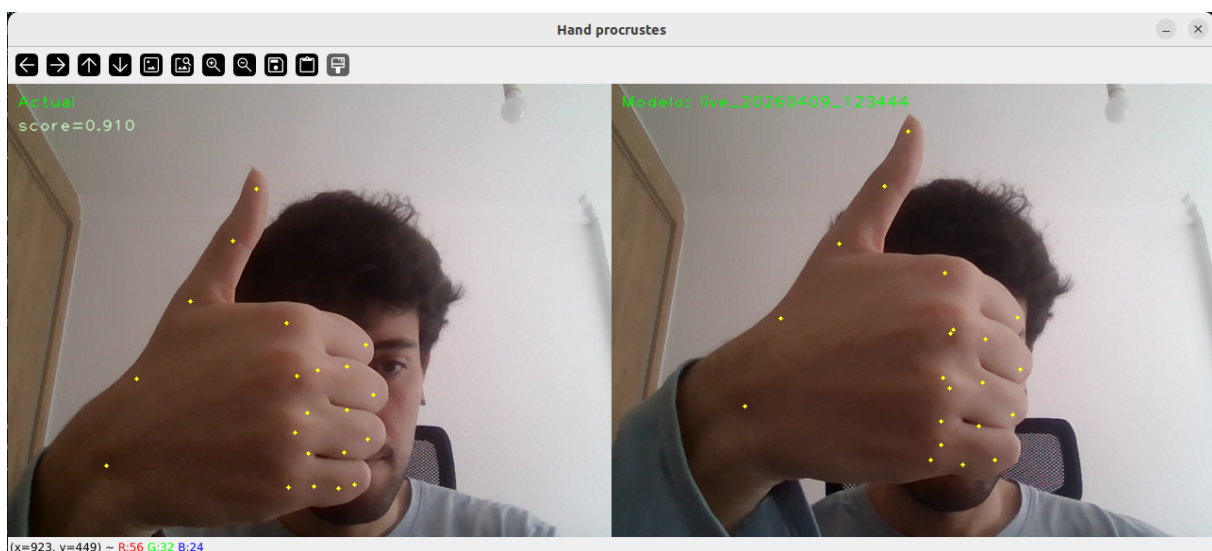


Figura 14: Clasificación de un gesto de mano. Se observa cómo el sistema detecta correctamente el pulgar hacia arriba.

5.5. Método 3: Coincidencia de puntos clave mediante SIFT

Este método emplea el algoritmo **SIFT** (*Scale-Invariant Feature Transform*) para identificar objetos basándose en sus detalles visuales locales. Es la técnica más robusta del sistema para reconocer elementos planos con gran cantidad de textura, como carátulas de videojuegos, portadas de libros o cuadros.

5.5.1. Extracción de Características y Descriptores

A diferencia de los métodos globales, SIFT identifica puntos de interés (keypoints) que son invariantes a cambios de escala, rotación e iluminación.

- **Limitación de rasgos:** Se ha configurado el detector para extraer un máximo de 500 puntos por imagen (`nfeatures=500`) con el fin de mantener un rendimiento fluido en tiempo real.
- **Descriptores:** Para cada punto, se genera un descriptor vectorial que captura la distribución del gradiente local, permitiendo una comparación matemática precisa entre el fotograma y el modelo.

5.5.2. Proceso de Clasificación

La clasificación se realiza comparando los descriptores del fotograma actual contra los de cada modelo mediante un emparejador de fuerza bruta (`BFMatcher`).

- **Búsqueda de vecinos:** Se emplea la función `knnMatch` con $k = 2$ para encontrar las dos mejores coincidencias para cada punto.
- **Filtro de Ratio de Lowe:** Para eliminar correspondencias erróneas o ambiguas, se aplica un umbral donde la distancia del mejor candidato debe ser inferior al 75 % de la distancia del segundo mejor candidato.
- **Métrica de Similitud:** El sistema devuelve tanto el número total de coincidencias válidas como un porcentaje relativo al total de puntos detectados en el modelo original, proporcionando una medida de confianza muy intuitiva.

5.5.3. Visualización de Correspondencias

Se ha integrado una visualización avanzada mediante `cv.drawMatches` que dibuja líneas verdes conectando los puntos emparejados entre la cámara y el modelo ganador. Esta herramienta permite verificar visualmente la calidad del reconocimiento y cómo el algoritmo gestiona las distorsiones de perspectiva.

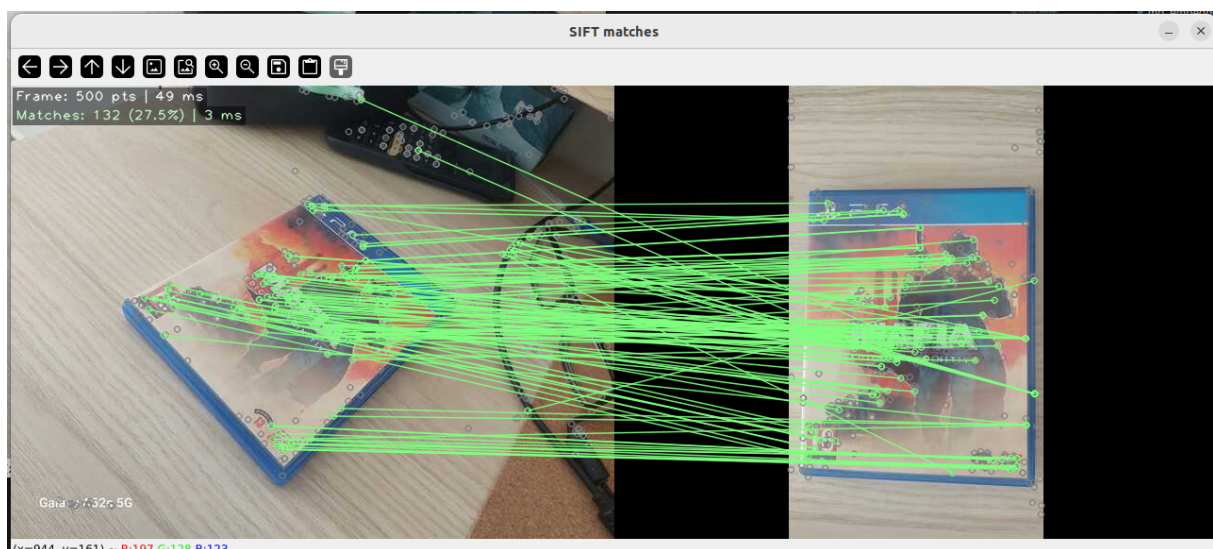


Figura 15: Reconocimiento de una carátula de videojuego mediante SIFT. El sistema detecta 132 coincidencias válidas (27.5 % de similitud), manteniendo el rastreo a pesar de la inclinación del objeto.

6. Deep Learning

mirar la carpeta **code/DL**, concretamente el fichero de jupyter.

7. Rectificación

El objetivo de este ejercicio es deshacer la deformación de perspectiva que sufre la imagen de un plano para poder medir distancias reales directamente sobre la imagen original. El principio se basa en que, si se conocen las coordenadas reales de al menos cuatro puntos visibles en la imagen, es posible estimar la transformación geométrica (homografía) que los relaciona y utilizarla para convertir cualquier par de píxeles clicados en una distancia real.

7.1. Marco teórico: Homografías

Cuando una cámara fotografía un plano, la relación entre las coordenadas de imagen (u, v) y las coordenadas reales (x, y) sobre ese plano es una **homografía** o transformación proyectiva, representada por una matriz H de tamaño 3×3 que actúa sobre coordenadas homogéneas:

$$\begin{pmatrix} \tilde{u} \\ \tilde{v} \\ \tilde{w} \end{pmatrix} = \begin{pmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}, \quad (u, v) = \left(\frac{\tilde{u}}{\tilde{w}}, \frac{\tilde{v}}{\tilde{w}} \right).$$

La matriz tiene 9 coeficientes pero el factor de escala global es arbitrario, por lo que tiene **8 grados de libertad**. Como cada punto del plano aporta 2 ecuaciones, se necesitan al menos **4 puntos de correspondencia** no colineales para determinarla de forma única. En este ejercicio se estima la homografía *inversa*, de coordenadas de imagen a coordenadas reales del plano ($H : \text{imagen} \rightarrow \text{real}$). Esto permite transformar cualquier píxel clicado (u, v) en su posición real (x, y) y calcular la distancia euclídea entre dos puntos como:

$$d = \|H p_1 - H p_2\|,$$

donde $p_1, p_2 \in R^2$ son los píxeles seleccionados. La homografía se estima a través de la función `cv.findHomography`.

Vista rectificada

Adicionalmente, para verificar visualmente que la estimación es correcta, se genera una imagen rectificada (vista fronto-paralela del plano). Se compone la homografía con un escalado s y un desplazamiento de origen para que el resultado quepa en un canvas positivo:

$$H_{\text{rect}} = \underbrace{\begin{pmatrix} s & 0 & 0 \\ 0 & s & 0 \\ 0 & 0 & 1 \end{pmatrix}}_S \cdot \underbrace{\begin{pmatrix} 1 & 0 & -x_{\text{mín}} \\ 0 & 1 & -y_{\text{mín}} \\ 0 & 0 & 1 \end{pmatrix}}_T \cdot H,$$

donde $(x_{\text{mín}}, y_{\text{mín}})$ es la esquina inferior del área real visible y s es el factor de escala (píxeles por unidad real). Aplicando H_{rect} a la imagen original con `cv.warpPerspective` se obtiene la vista rectificada, donde la distancia entre dos puntos en píxeles dividida por s da directamente la distancia real.

7.2. Descripción del script

El script `code/rectfificacion/rectificacion.py` se ejecuta en dos modos y se lanza con el argumento `--loop` para mantener la imagen estática abierta en bucle:

- **Modo `--pick`:** el usuario hace clic sobre los puntos de referencia visibles en la imagen. Al terminar, el programa solicita por terminal las coordenadas reales de cada punto e introduce el par de valores por cada uno, y guarda automáticamente el fichero `ref-<nombre>.txt`. De esta forma no es necesario conocer de antemano las coordenadas de píxel.
- **Modo `--ref`:** cargando el fichero de referencia ya creado, el script calcula la homografía y abre una ventana interactiva donde el usuario puede hacer clic en dos puntos de la imagen original para obtener su distancia real. La etiqueta se muestra superpuesta sobre la imagen original con una línea roja entre los puntos. Opcionalmente, con la tecla `r`, se abre la vista rectificada donde también se puede comprobar las operaciones previas.
 - **Click izquierdo:** marca un punto de medición. Al marcar dos puntos, se dibuja una línea roja entre ellos y se muestra la distancia real calculada en blanco.
 - **Click derecho:** borra los puntos de medición marcados.
 - **Tecla `r`:** muestra u oculta la vista rectificada (fronto-paralela). En esta ventana también se pueden marcar puntos para medir distancias directamente sobre el plano rectificado.
 - **Tecla `s`:** guarda una captura de la imagen actual en disco.
 - **ESC / `q`:** cierra la aplicación.

En cada imagen debe existir un objeto de referencia, para la foto `coins.png` se utiliza un **carnet de conducir**, que sigue el estándar ISO/IEC 7810 ID-1 con dimensiones exactas de $8,56 \times 5,4$ cm. Al ser un objeto que siempre está disponible y cuyas medidas son conocidas con precisión, resulta ideal para este tipo de aplicaciones.

7.3. Resultado sobre coins.png

Para la imagen de las monedas se utilizó el carnet de conducir como objeto de referencia, marcando sus cuatro esquinas con `--pick` y asignando las coordenadas reales en centímetros. Con la homografía calculada, se midió la distancia entre las dos monedas.

La Figura 16 muestra la vista rectificada: el plano de la mesa se observa frontalmente y el carnet aparece perfectamente rectangular, lo que confirma que la homografía es correcta. La medición realizada en la vista rectificada da una distancia de **10,5 cm** entre las dos monedas.

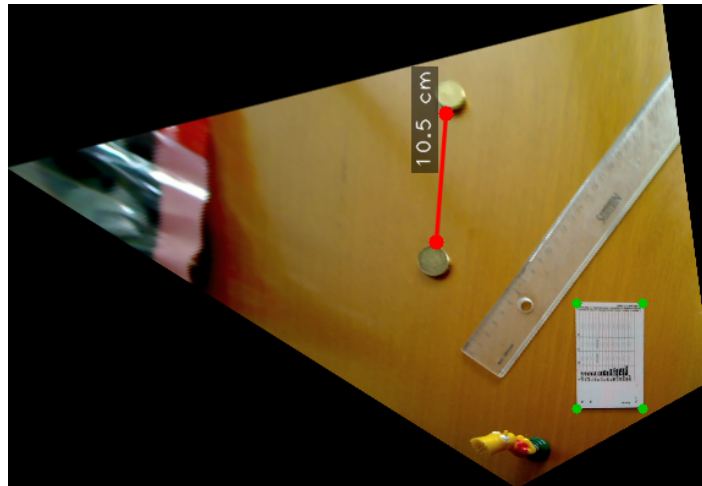


Figura 16: Vista rectificada de la escena con las monedas. El plano de la mesa se ve frontalmente. El carnet (esquinas en verde) aparece rectangular, confirmando la correcta estimación de la homografía. La distancia medida entre las monedas es de 10,5 cm.

La Figura 17 muestra la misma medición realizada sobre la imagen original, tal como exige el enunciado: los puntos se marcan con clics sobre la imagen con perspectiva y la distancia real se calcula transformando sus coordenadas al plano real mediante la homografía.

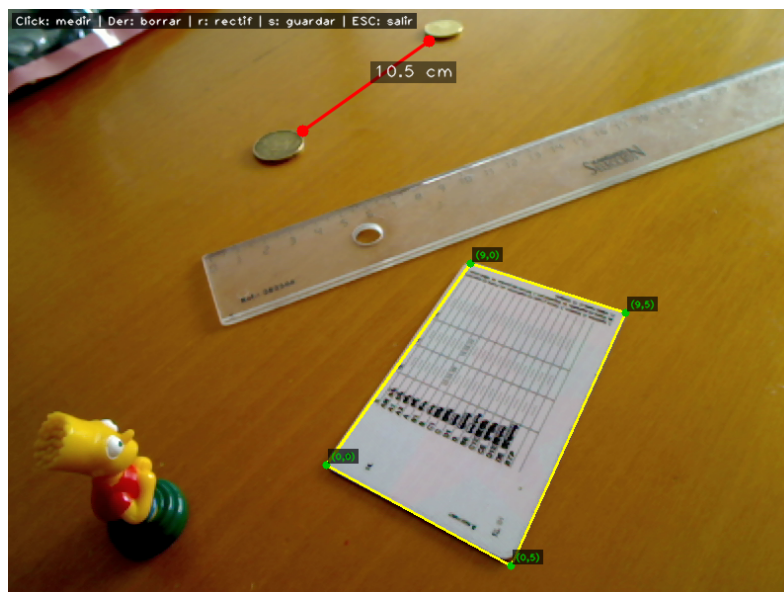


Figura 17: Medición de la distancia entre las monedas sobre la imagen original. Los puntos de referencia del carnet aparecen marcados en verde. La distancia obtenida es de 10,5 cm.

7.4. Verificación con imágenes propias

Para verificar el sistema con imágenes propias se preparó una escena con distintos objetos sobre una superficie plana. Como objeto de referencia se eligió la caja del videojuego *Mafia*, un rectángulo de dimensiones conocidas: **17 cm** de largo y **13,5 cm** de ancho. Al igual que con el carnet, sus cuatro esquinas se marcaron con el modo `--pick` y se asignaron sus coordenadas reales. La homografía se calculó con 4/4 inliers, lo que indica que los

puntos de referencia son consistentes con un plano y que la transformación se ha estimado correctamente.

Una vez establecida la referencia, se realizaron mediciones sobre distintos objetos de la escena. La Figura 18 muestra la medición de un lado del cartabón sobre la imagen original con perspectiva, donde el sistema estima **22,5 cm**. Este resultado es especialmente interesante porque el cartabón, al ser un triángulo con solo tres vértices, no puede usarse directamente como referencia (se necesitan cuatro puntos no colineales para la homografía), pero sí puede medirse con el sistema una vez que la referencia es otro objeto de la escena.

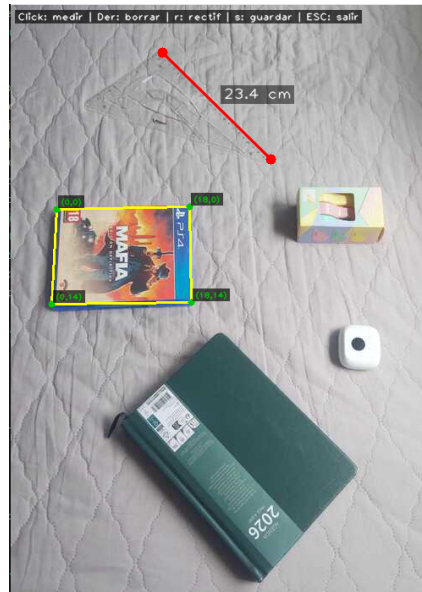


Figura 18: Medición sobre imagen propia. El juego actúa como referencia y la medida del lado se obtiene a partir de la imagen original.

A continuación, en la vista rectificadora de la misma escena (Figura 19), se midió la distancia entre un paquete de tipex y unos AirPods, obteniendo **11,4 cm**.

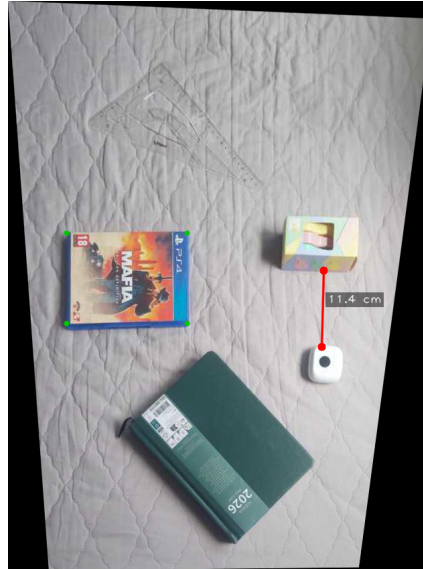


Figura 19: Vista rectificada de la escena propia. La perspectiva ha sido eliminada y el plano se observa frontalmente. La distancia entre el paquete de tipex y los AirPods es de 11,4 cm.

Para validar el resultado, se midió físicamente con un metro la misma distancia, obteniendo **11 cm**, tal como se aprecia en la Figura 20. El error absoluto es de 0,4 cm, lo que supone un error relativo aproximado del **3,6 %**. Este pequeño error es atribuible a la imprecisión en el marcado manual de los puntos de referencia y a la distorsión de la lente.

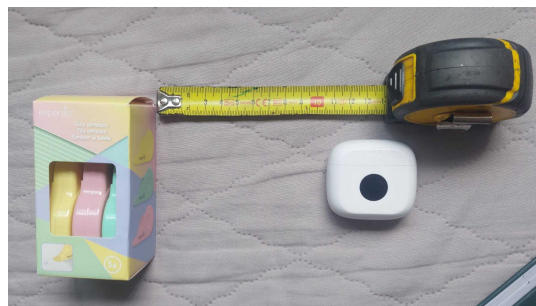


Figura 20: Validación física con metro. La distancia real entre el paquete de tipex y los AirPods es de 11 cm, frente a los 11,4 cm estimados por el sistema (error del 3,6 %).

7.5. Limitaciones

El método es válido únicamente para objetos situados en el mismo plano que los puntos de referencia. Objetos elevados (como la cara superior de un cubo) introducen errores de escala. Además, la precisión decrece conforme nos alejamos de los puntos de referencia, ya que los errores de marcado se amplifican con la perspectiva. Por último, se necesitan al menos cuatro puntos no colineales, por lo que objetos con solo tres vértices visibles (como un cartabón) no son suficientes por sí solos para estimar la homografía.

8. OPCIONAL: Carnet UMU

mirar la carpeta `code/opcional-UMU`, concretamente el fichero de jupyter y script `card.py`.

9. OPCIONAL LK: Dirección de movimiento y velocidad angular

El objetivo es extender el tracker de Lucas-Kanade (`lk_track.py`) para determinar la dirección dominante de movimiento de la cámara (UP, DOWN, LEFT, RIGHT, FORWARD, BACKWARD) y estimar su velocidad angular en grados por segundo.

9.1. Marco teórico: Lucas-Kanade

El método de **Lucas-Kanade** es un algoritmo de flujo óptico que estima cómo se desplazan los puntos de la imagen entre dos fotogramas consecutivos. Se basa en dos hipótesis: la *constancia de brillo* (la intensidad de un punto no cambia al moverse) y la *coherencia local* (todos los píxeles de una pequeña ventana se mueven igual). Bajo estas hipótesis, el vector de flujo (u, v) satisface:

$$I_x u + I_y v + I_t = 0,$$

donde I_x, I_y, I_t son las derivadas espaciales y temporal de la imagen. Al aplicar esto a todos los píxeles de la ventana se obtiene un sistema sobredeterminado que se resuelve por mínimos cuadrados. La versión **piramidal** (`calcOpticalFlowPyrLK`) aplica LK en múltiples escalas para manejar movimientos grandes.

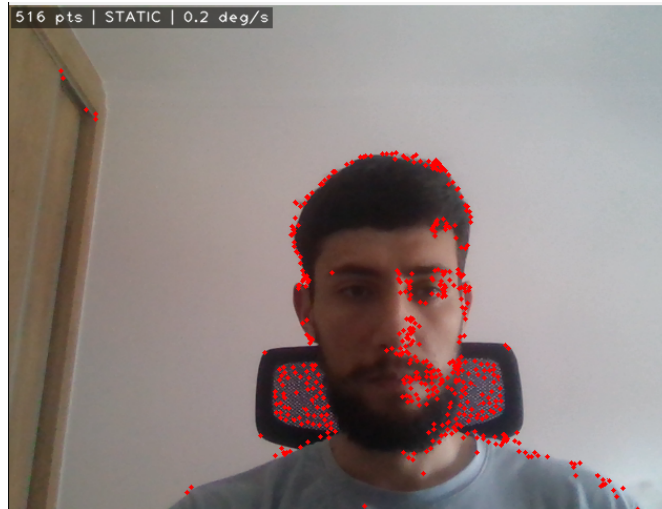


Figura 21: Tracker LK en marcha: trayectorias rojas de los puntos rastreados y flecha verde del flujo medio amplificado.

9.2. Estimación de la dirección de movimiento

A partir de las correspondencias (p_0^i, p_1^i) se estima una **transformación afín similar** (rotación + escala + traslación) con **RANSAC** (*Random Sample Consensus*), técnica que prueba muchas hipótesis aleatorias con subconjuntos mínimos de puntos y se queda con la que tiene más correspondencias compatibles (*inliers*):

$$M = \begin{pmatrix} s \cos \theta & -s \sin \theta & t_x \\ s \sin \theta & s \cos \theta & t_y \end{pmatrix}, \quad p_1 \approx M p_0.$$

Los parámetros extraídos codifican distintos movimientos de la cámara. La clave es que la escena se desplaza en la imagen en sentido **contrario** al movimiento de la cámara: si la cámara va a la derecha, los objetos parecen moverse a la izquierda ($t_x < 0$).

Dirección cámara	Condición
LEFT	$t_x > 0$ (escena se desplaza a la derecha)
RIGHT	$t_x < 0$ (escena se desplaza a la izquierda)
UP	$t_y < 0$ (escena se desplaza hacia abajo)
DOWN	$t_y > 0$ (escena se desplaza hacia arriba)
FORWARD	$s > 1$ (escena se expande)
BACKWARD	$s < 1$ (escena se contrae)

Cuadro 1: Dirección de movimiento de la cámara según los parámetros de la transformación afín.

La dirección dominante se selecciona convirtiendo los tres componentes a píxeles equivalentes ($|s - 1| \cdot W$ para el zoom) y tomando el de mayor magnitud por encima de un umbral mínimo.



Figura 22: Las seis direcciones detectadas con la etiqueta y la flecha de flujo correspondientes.

9.3. Estimación de la velocidad angular

La transformación afín M codifica la rotación en el plano de la imagen. El ángulo θ se extrae directamente de sus coeficientes:

$$\theta = \arctan\left(\frac{M_{10}}{M_{00}}\right).$$

Dividiendo por el tiempo entre fotogramas se obtiene la **velocidad angular aproximada**:

$$\omega = \frac{\theta}{\Delta t} \quad [\text{deg/s}].$$

Para estimar la velocidad angular de rotaciones fuera del plano (panorámica o inclinación) sería necesario recurrir al **modelo pinhole**: una rotación de ángulo α produce un desplazamiento aparente $t \approx f \tan \alpha$ píxeles, con f la distancia focal. Sin calibración de la cámara no se puede determinar f con precisión, por lo que esta componente queda fuera del alcance de esta estimación.



Figura 23: Captura durante una rotación rápida¹: la velocidad angular elevada aparece en el texto superpuesto junto a las trayectorias con giro visible.

9.4. Script lk_motion.py

Dentro de la carpeta (code/opcional-lk) podemos ejecutar lo siguiente,

```
1 python lk_motion.py # webcam por defecto
2 python lk_motion.py --dev=video.mp4
```

La ventana muestra en tiempo real las trayectorias, la flecha de flujo medio amplificada y el texto con la dirección dominante y la velocidad angular en deg/s.

¹Comando: `python lk_motion.py --dev="$(yt-dlp -f worst --get-url Y0CrXGUBkgk)"`

10. OPCIONAL: Colmap vs Vggt

mirar la carpeta `code/opcional-colmap`, concretamente el fichero de jupyter.

11. OPCIONAL: Realidad Aumentada, desplazamiento de una pelota

mirar la carpeta `code/opcional-ra`, concretamente el fichero de jupyter.

12. Conclusión

La presente entrega ha permitido validar la integración efectiva entre las técnicas de **visión artificial** y los modelos de **Deep Learning** para la resolución de problemas reales. A través de los distintos bloques desarrollados, se han alcanzado las siguientes conclusiones:

- **Precisión Métrica:** La calibración de cámara basada en el modelo *pinhole* ha sido fundamental para aplicaciones de metrología, logrando un error relativo inferior al 2,15 % en mediciones físicas reales.
- **Monitorización y Seguimiento:** El análisis de tráfico y el detector de movimiento han probado la robustez del algoritmo *MOG2*. Se ha verificado que la congelación del fondo permite gestionar retenciones de tráfico, mientras que la integración con *YOLO* garantiza un filtrado de eventos selectivo y el cumplimiento de la privacidad mediante anonimización.
- **Segmentación con Deep Learning:** El ejercicio con la arquitectura **U-Net** ha demostrado la potencia de la segmentación semántica para aislar regiones de interés. Se ha constatado que, aunque el modelo es preciso en entornos conocidos, la generalización en escenarios nuevos depende directamente de la diversidad de las imágenes de entrenamiento.
- **Modularidad en el Reconocimiento:** El clasificador modular ha evidenciado que el éxito del reconocimiento depende de la técnica empleada: desde la invariancia geométrica de *Procrustes* para gestos, hasta la robustez local de *SIFT* para texturas complejas y el uso de *embeddings* para contenido semántico.
- **Reconstrucción 3D:** La comparación entre el pipeline clásico de Structure-from-Motion (COLMAP) y el transformer *vggt* ha puesto de manifiesto la superioridad de los enfoques basados en aprendizaje profundo: con solo 15 imágenes, *vggt* produce una reconstrucción densa notablemente mejor que COLMAP con 180 fotogramas, especialmente en objetos con poca textura superficial donde SIFT no encuentra correspondencias fiables.
- **Realidad Aumentada:** La integración de calibración intrínseca, estimación de pose y homografía del plano $z = 0$ ha permitido posicionar una esfera virtual sobre un marcador planar con movimiento suave y rodamiento físicamente correcto, demostrando cómo los conceptos teóricos de la asignatura se aplican directamente en un sistema AR interactivo.

En definitiva, la combinación de descriptores clásicos, redes neuronales convolucionales y técnicas de geometría proyectiva constituye un ecosistema sólido para la creación de sistemas de visión inteligentes, fiables y adaptables a diferentes necesidades computacionales.